

AI Study Companion — System Architecture & Technical Design

Candidate Challenge Edition v3.0 • Full Stack AI Engineer

Candidate: Sairam Anakala

Repository: https://github.com/sairam0043/AI_Prof

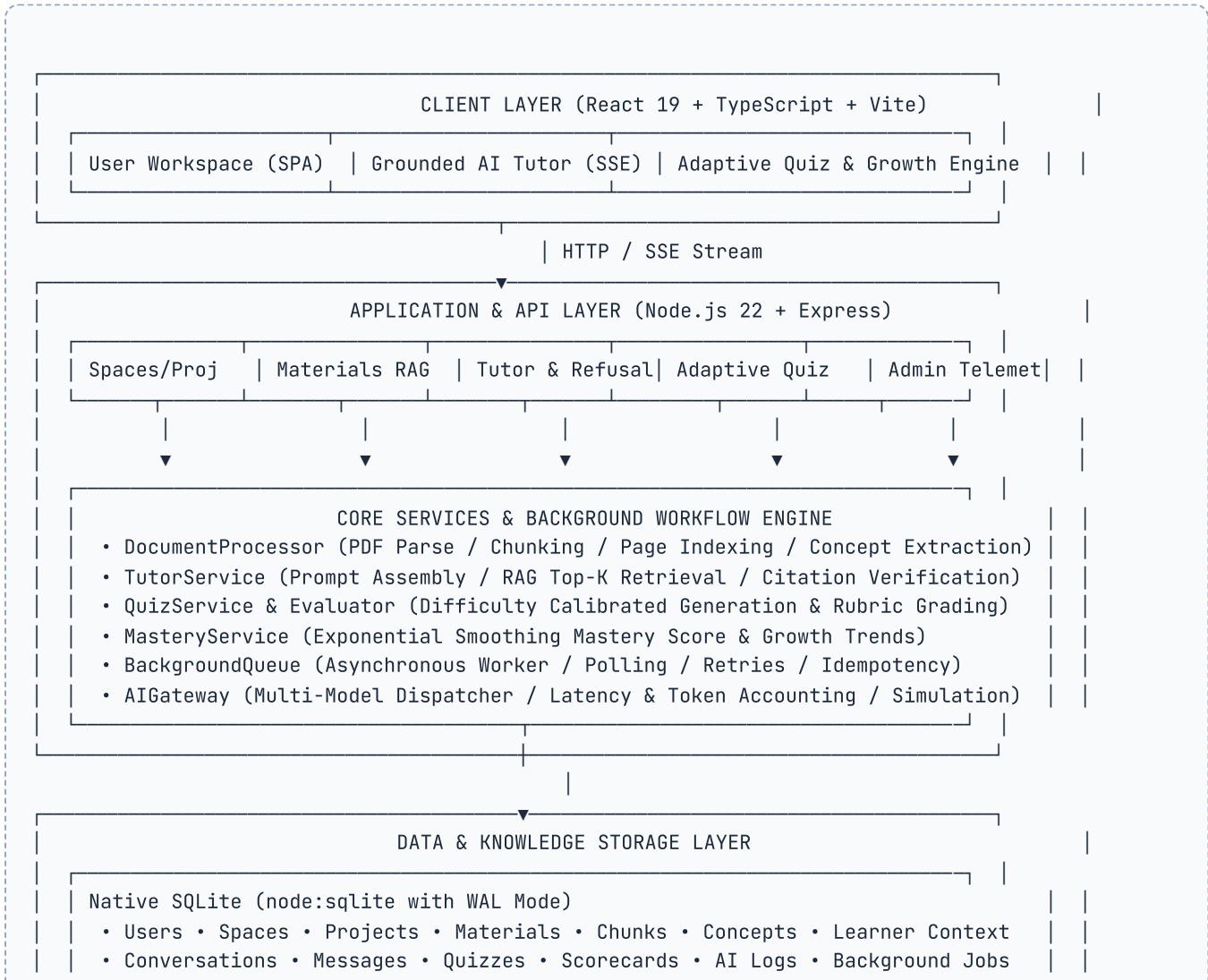
Target Role: Full Stack AI Engineer Intern

Architecture Pattern: Modular Monolith / Multi-Tenant Isolation

1. Executive Summary & System Goals

AI Study Companion is a persistent, contextual, and measurable AI-powered learning workspace. Rather than functioning as a disconnected chatbot, the platform unifies document indexing, grounded AI tutoring, adaptive quizzes, multi-factor open-ended assessments, concept mastery tracking, and event-driven recommendations into a continuous learning loop.

2. High-Level Architecture & End-to-End Data Flow



```
In-Memory Cosine Vector Index: Partitioned strictly by project_id
```

3. Key Architectural Components

3.1 Data Isolation & Scoping Hierarchy (User → Space → Project)

The platform enforces strict project-level data isolation. A user can manage multiple *Spaces* (e.g. Machine Learning, Distributed Systems), each hosting multiple focused *Projects* with distinct learning goals. All database queries, vector indexes, conversation histories, and mastery models require explicit `project_id` filters to prevent cross-domain data leakage.

3.2 Asynchronous Document Processing Pipeline

PDF document uploads do not block HTTP request cycles. Upon upload:

1. Document record is initialized with state `queued` and dispatched to `background_jobs`.
2. The background worker polls jobs every 2 seconds and transitions state to `processing`.
3. **Text & Structure Extraction:** Preserves page boundaries and headings via `pdf-parse`.
4. **Semantic Chunking:** Partitions text into 300–600 token chunks retaining page numbers (`page_number`) for exact citation traceability.
5. **Vector Indexing:** Embeds chunks and updates the project's cosine similarity search index.
6. **Automated Concept Extraction:** Synthesizes key domain concepts and seed mastery levels.
7. Job status is marked `ready`.

3.3 Grounded AI Tutor & Anti-Hallucination Refusal Guardrail

The AI Tutor answers questions by prioritizing verified source material through a rigorous 4-step context pipeline:

- **Context Assembly:** Project Learning Goal + Top-K Retrieved Document Chunks + Concept Mastery Profile + Learner Weaknesses + Recent Conversation Window.
- **Citation Badge Injection:** Whenever an answer utilizes document evidence, it attaches verified citations in the format: `[Source: Document Title – Page X]`.
- **Anti-Hallucination Refusal Guardrail:** If the user asks an out-of-domain or unsupported question (e.g., cooking recipes in an ML workspace), the system detects zero grounding confidence, refuses to fabricate answers, and steers the learner back to available curriculum concepts.

3.4 Adaptive Quiz & Multi-Factor Assessment Engine

Rather than naive wrong→easy / correct→hard heuristics, the quiz engine evaluates the learner's multi-dimensional state:

- **Question Selection:** Analyzes concepts marked *Requiring Attention* (<50% mastery) or frequent mistakes.
- **Open-Ended Conceptual Evaluation:** LLM evaluates subjective student answers against 4 distinct metrics:
 - **Understanding Score (0–100%)**
 - **Technical Accuracy (0–100%)**
 - **Key Concepts Covered Checklist**
 - **Missing Nuances / Actionable Guidance**

3.5 Concept Mastery Growth Model

Mastery is computed dynamically using an exponential smoothing update formula:

$$\text{New Mastery} = (0.60 * \text{Previous Mastery}) + (0.40 * \text{Latest Assessment Performance})$$

Concepts are classified into three actionable status tiers:

-  **Improving:** Mastery ≥ 75% and positive performance trajectory.
-  **Stable:** Mastery between 50% and 74%.
-  **Requiring Attention:** Mastery < 50% or recent repeated mistakes.

4. Important Architectural Decisions & Trade-Offs

Decision	Selected Approach	Rationale	Alternative Considered
Database Engine	Native SQLite (<code>node:sqlite</code>) with WAL Mode	Zero native compilation errors, microsecond synchronous reads, ACID transactions, single-file portability.	PostgreSQL / MongoDB (unnecessary infrastructure overhead for 3-4 day challenge).

Vector Retrieval	In-Memory Cosine Similarity Index scoped by Project	Instant vector search without requiring external Pinecone / Qdrant subscriptions; easily scales to tens of thousands of chunks.	External Vector DB SaaS (introduces network latency and API key dependencies).
AI Gateway	Gemini 1.5 Flash + Fallback Simulation Engine	Gemini 1.5 provides high token throughput and low latency; built-in simulation allows regression tests to run offline.	Pure OpenAI (higher API costs and potential rate limits).
Full-Stack Serving	Unified Production Express Serving React SPA	Eliminates CORS complexities, simplifies Render deployment to a single web service.	Split Frontend/Backend Hosting (adds deployment configuration complexity).

5. Observability, Security & Production Readiness

- **Telemetry:** Every AI request logs `model`, `feature`, `prompt_tokens`, `completion_tokens`, `latency_ms`, `cost_cents`, and `status` into `ai_logs` table.
- **Data Protection:** Uploaded materials and context vectors are strictly isolated by `user_id` and `project_id`.
- **Security:** Environment variables and secrets are completely isolated from source code.